



Everything You Always Wanted to Know About Embedded Trace

Thomas B. Preußer, Accemic Technologies

Smitha Gautham, Abhi D. Rajagopala, and Carl R. Elks, Virginia Commonwealth University

Alexander Weiss, Accemic Technologies

Decades of advances in computer architecture, software-intensive applications, and system integration have created significant challenges for embedded systems designers and test engineers. This article explains embedded trace as an essential technology in the testing and debugging toolbox. It highlights its capabilities, limitations, and opportunities.

As embedded systems have become ubiquitous, we increasingly rely on their functionality to perform tasks that were once laborious, costly, tedious, or even unattainable. Advanced embedded system technology has been one of the key driving forces behind the rapid growth of cyberphysical system (CPS) applications. Applications in robotics, autonomous systems, telemedicine, and avionics are examples of software-intensive systems taking on increasingly complex software functionality. For

example, a 2018 midpriced automobile relies on about 100 million lines of code distributed over dozens of embedded processors. This rising trend in software-intensive systems, in turn, requires advanced hardware platforms, such as systems on chip (SoCs) or multicore heterogeneous processors. The confluence of advanced embedded computing hardware and software-intensive systems creates challenges for testing and verification activities, especially for systems that demand high levels of reliability, safety, and security.

As noted by Weiss et al.,¹ advanced embedded applications can breed convoluted software anomalies, such as heisenbugs, which evade debugging by changing their

Digital Object Identifier 10.1109/MC.2021.3098965
Date of current version: 14 February 2022

behavior under intrusive probing. The ability to detect, debug, and isolate such faults efficiently and effectively is essential for a cost-effective system verification. A key prerequisite is the observability of execution flows, memory access patterns, and data-flows. Most often, program behavior is observed by software instrumentation including breakpoints and printf-style debugging. These approaches introduce probes and, thus, violate the desirable property of nonintrusiveness. Especially in complex embedded systems with multithreaded programs and real-time requirements, software instrumentation can result in timing and synchronization issues that alter the observable and, in fact, required program behavior.²

A promising technology addressing the program observability is embedded trace (ET). ET is best thought of as a set of real-time streams of software execution data nonintrusively emitted by a processor. It can reveal memory access patterns, the control, and the data flow of the executing software. Although ET is an integral part of almost all modern processors, it is virtually unknown to application engineers, test engineers, and project managers. Their fundamental understanding of ET is very much needed at this time, as it can be vital for verifying and debugging software, especially in critical systems.

Flight software

Civil aviation certification authorities mandate to test what you fly and fly what you test. This mandate ensures that safety claims are based on actual flight software and avoids the danger of activating dormant heisenbugs by the removal of test code. However, it also implies that any test instrumentation

must remain part of critical flight software. Thus, instrumentation comes at a permanent cost of power consumption and system performance. Also, execution data cannot be collected spontaneously in operation as long as it requires an instrumentation that must undergo the whole test and certification process. ET is the solution that enables both the nonintrusive monitoring of tests and the flexible collection of execution data from unmodified lean release code without instrumentation.

Automotive electronic systems

Dozens of vendor-specific electronic control units (ECUs) provide the smooth operability and functionality of modern automobiles. These ECUs and their integrated software components are often characterized as black boxes hiding their proprietary implementation behind a functional application programming interface. The analysis of timing integration problems often requires detailed insights into the execution behavior within the system context. ET enables the online observation of the executing ECU software even without requiring access to its source code.

This article provides insights into the technology of ET for embedded systems application engineers and for the testing and verification community. It highlights the capabilities and limitations of ET by detailing what information it delivers, what is needed for its useful interpretation, and what opportunities it creates in simplifying and streamlining test and debug procedures. Understanding the fundamentals of ET and its application to testing enables embedded CPSs that are more reliable, safer, and more secure.

TRADITIONAL SOFTWARE INSTRUMENTATION

A widely established observation technique is software instrumentation. The application software is modified to produce the desired information, such as control-flow indicators, via standard output channels. The used mechanisms include manual printf debugging as well as automated source code and binary instrumentation.

All of these approaches impact the timing behavior and the memory footprint of an application massively. Instrumentation often also entails the phantom synchronization between control flows by the arbitration of shared output channels. In any case, the behavioral equivalence of instrumented and noninstrumented code is compromised, challenging the validity of functional tests. These consequences preclude the temporary instrumentation of already timing-critical control loops for the purpose of higher level integration and system tests. They are also the reason why safety-critical software is required to repeat its test and certification process upon instrumentation. Less critical application domains sometimes choose to mitigate the underlying observability challenge by demonstrating test coverage and functional correctness in two separate test runs: the first monitoring the execution of instrumented code and the second using noninstrumented release code. This repetition is costly. The growing system complexities erode the confidence in this approach as it is prone to hiding heisenbugs. Despite its serious drawbacks, software instrumentation is still widely used—often due to the lack of a handy alternative.

ET

Motivation

In the olden days, processors were slow (some 10 MHz) and only had external program memory. Eavesdropping on the memory bus, as shown in Figure 1, was sufficient to observe the memory accesses by the CPU. Following the program execution along the fetched instructions was trivial.

CPUs have become faster, and their architectures have been changing dramatically. Caching instructions and data near the CPU, inside the same chip, has removed the individual memory access from external visibility. SoCs with readily integrated RAM

and Flash memory even exacerbate this issue. Finally, the current trajectory toward integrating more and more CPU cores into a single package further complicates the attribution of the remaining observable external behavior to one of the concurrent control flows scheduled across truly parallel cores. All of these advances in processor architecture and system integration have mandated ET as a radically new approach to observing what is happening inside the processor.

ET is implemented by functional units that make CPU activity observable. It faces a significant bandwidth challenge as monitoring a CPU comprehensively requires information about

the executed instructions and affected registers. For a 32-bit CPU, a naïve encoding would produce trace data of roughly 70 bits for each instruction, including its address as well as register and flag updates. This translates to a bandwidth demand of 70 Gbit/s when running at 1 GHz. This is far beyond an economically reasonable solution and demands that trace data be compressed.

Protocol overview

Actual trace implementations such as Arm CoreSight Program Flow Trace (PFT),³ Arm CoreSight Embedded Trace Macrocell (ETM),⁴ Intel Processor Trace (Intel PT),⁵ (Sec. 35) and Nexus 5001 Forum⁶ compress the transmitted control-flow trace vigorously. They inject relevant operational information to facilitate a highly efficient, context-aware compression. System observability can be further boosted by tracing the data flow. However, the implied trace data are much harder to compress. This results in significantly higher bandwidth requirements and, hence, more costly trace interfaces. Therefore, most implementations refrain from implementing this capability. We start elucidating the trace compression techniques with an overview over the most relevant trace messages as categorized by Table 1.

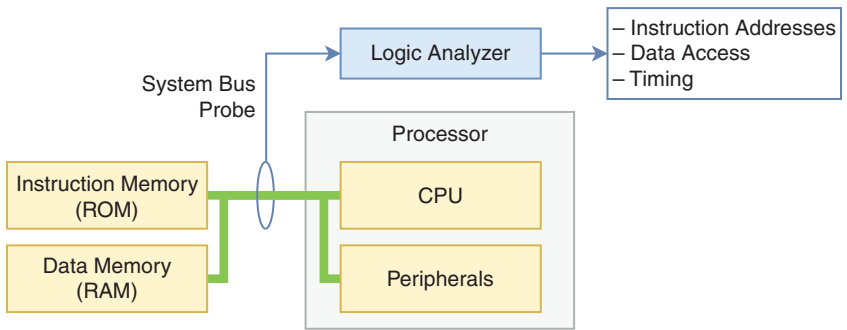


FIGURE 1. The olden days—all CPU activity is visible on the external bus.

TABLE 1. The most relevant trace message categories.

Control-flow messages	Timing messages
Synchronization	Timestamps
Branching	Cycle counts
Exceptions	Clock speed
Data-flow messages	Other messages
Memory addresses	Trace buffer overflows
Data values	Lightweight instrumentation

Control-flow messages. The continuous output of the program counter alone would consume significant trace bandwidth. This is avoided by 1) assuming that the executed application is known to the observer and 2) exploiting the default sequential execution of instructions. This compression strategy gives rise to specialized control-flow messages as described here.

Synchronization messages are generated in greater intervals, for example,

every few thousand messages. They alone establish a full application context and the program counter that serves as the reference point for the further interpretation of the trace data. The execution of the sequential code following this point is implied.

Branch messages communicate actual control-flow decisions. A single bit indicates whether a conditional branch has been taken. Branches not taken imply the sequential continuation of the execution. They do not require any further trace data. As the target address of taken direct branches can be inferred from the executing binary, only taken indirect branches trigger the generation of an alternate larger message that enables the observer to reconstruct the dynamically computed branch target address. The handling of unconditional direct branches is specific to the ET architecture. Some choose to establish posts in the control flow and produce taken execution indicators just as they do for executed conditional direct branches (for example, Arm PFT). Others pass over this clearly implied control flow without any representation in the emitted trace (for example, Intel PT).

Exception messages are generated for externally induced control-flow diversions such as by interrupts. They typically provide a hint on the nature of the exception, identify where the implied control flow was interrupted, and contain all information necessary to resume the control-flow reconstruction, either including or skipping the interjected handler depending on the trace configuration.

Note the vigorous reduction of the control-flow trace to the resolution of dynamic control-flow decisions. Its encoding is the other ingredient for pushing the average trace bandwidth

below a single bit of trace data per executed instruction. The decision indicators generated for conditional branches are aggregated into messages with a very small protocol overhead. Up to five (Arm CoreSight) or six (Intel PT) of them are transmitted within a single byte. Address transmissions, as after the executions of indirect branches, refer back to the preceding communicated instruction pointer so as to only encode a difference. This typically avoids the transmission of complete 32- or even 64-bit addresses.

Timing messages. An ET architecture may define message formats for conveying timing information. Between occasional transmissions of the wall clock, cycle counts indicate how many CPU clock cycles have elapsed since the last timing update. Observers can typically choose to receive cycle-count messages with each branch message, in programmable time intervals or not at all. This way, the significant trace bandwidth consumption by high-frequency timing messages can be balanced against other desired trace quality properties.

Data-flow messages. Data trace is difficult to compress. Depending on the trace architecture, the address of a data access, the transmitted value, and the type of access may be communicated. In a 32-bit system, each access can result in a trace message of more than 60 bits in length. Due to its high bandwidth requirements, data trace is not available in all implementations and otherwise regularly limited. For instance, it may be constrained to designated address regions or to producing partial information such as the addresses of write accesses only.

Other messages. There are a number of other trace message types for transmitting contextual and auxiliary information. They may allow the operating system to communicate context switches, report internal trace buffer overflows, or facilitate a very lightweight, trace-based code instrumentation.

Illustrating trace example

For an illustration of the vigorous compression of the control-flow trace, consider the example of Figure 2. It comprises a simple iteration over some input for counting all occurrences of the value 42. Assume that the compiler has generated the annotated x86_64 assembly for the code of interest. Two possible Intel PT trace observations are tabulated.

Every now and then, a synchronization block is emitted. It is opened by a packet stream boundary (PSB) message whose encoding can be uniquely detected even in an unsynchronized trace stream. It is a point, at which a decoder can always tap in safely. From there on, a decoder has to slice the stream into messages on the basis of a variable-length prefix code. The synchronization block is terminated by a PSBEND message. In between, the state is established that is needed for the context-aware compression of the subsequent actual trace. This ensures that not only the decoding of messages but also the interpretation of their data are completely synchronized. Typical constituents of this state are a time basis (TSC, CBR), a process identifier (PIP), and the current program counter (FUP). Assume that the latter positions the observed execution just at preparing `arg` as the argument to `count42()`.

The following actual trace only communicates dynamic control-flow decisions that cannot be inferred from the assembly. Particularly, the control transfer into function count42 is simply implied. The first event identified in the trace is the decision triggered by the comparison with 42 in this function. Receiving a “not-taken” indication in our example, we can infer that the encountered value was not a 42.

For the sake of continuing our reconstruction of the executed control flow, we observe that the next non-sequential control-flow transition is prompted by the function return. Two options exist for its trace representation. The trace source may always

choose the resolved encoding. Treating the return just like an indirect branch, it communicates the return address as an explicit reentry target. This communication is, indeed, compressed as it conveys only a patch against the most recently transmitted instruction pointer, which, in our example, was established by the FUP message. The trace then continues by indicating that the conditional branch implementing the loop was taken.

Observe that the transmission of the instruction pointer is costly even though it is regularly just a small patch against the last established value. In the case of function return addresses,

a shadow call stack maintained by the decoder is typically able to provide the needed value as well. Such a widely supported return compression allows to turn the transmission of the program counter into a single-bit “taken” indicator. To ensure that the trace can be interpreted correctly, this very effective compression is predicated on a few conditions: 1) the actual return address must match the address that was pushed by the corresponding call, 2) the return was not separated from this call by a synchronization block, and 3) the return address never sunk below an architecture-defined stack depth during the execution of the enclosed

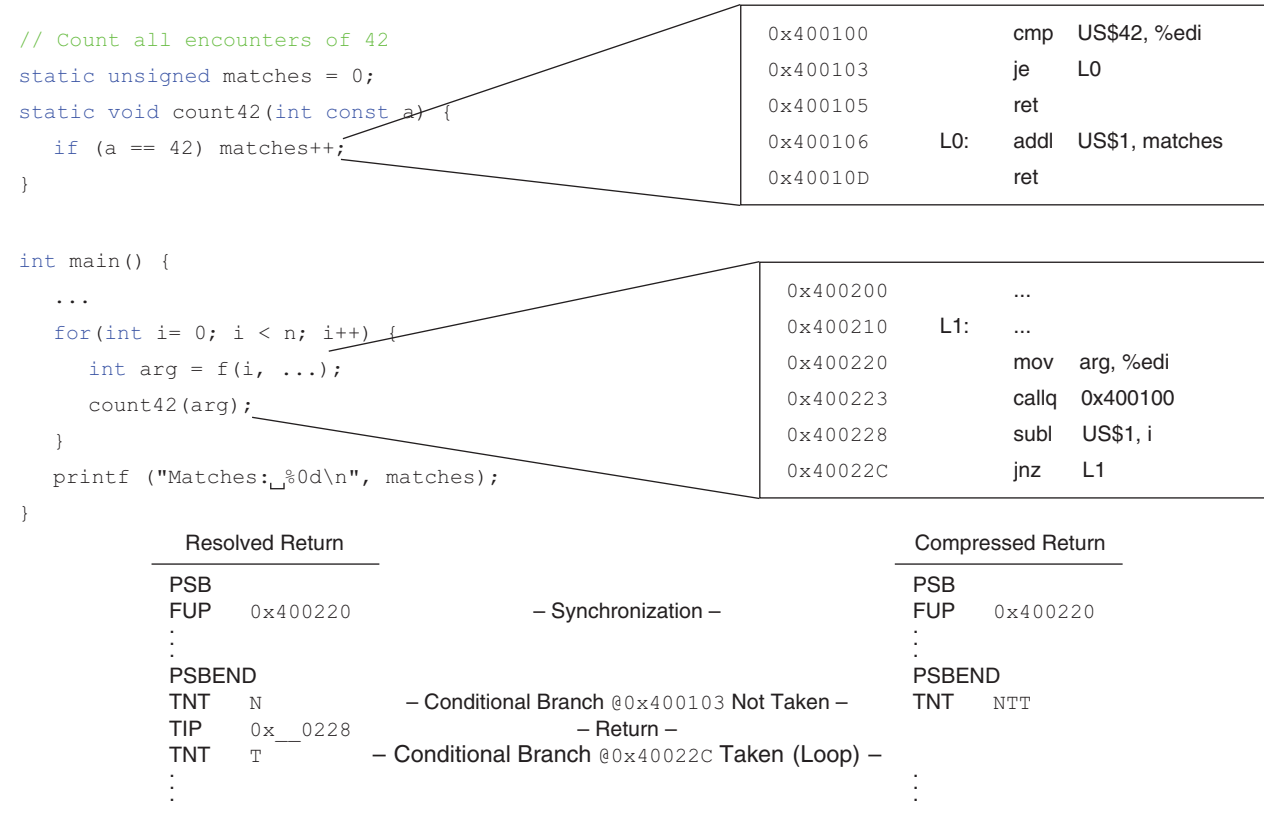


FIGURE 2. An illustrating execution trace example.

call hierarchy. This guarantees that a decoder, even if freshly synchronized on the trace, will have seen all information needed to resolve a compressed return. Finally, note the additional effect in the trace example that the sequence of “taken” and “not-taken” indicators can now be aggregated into a single, one-byte message for an improved payload utilization.

ET evolution

As system observability was challenged by an advancing integration and faster clock speeds, monitoring capabilities had to become an aspect of the system design itself. While bond-out chips are able to expose internal bus activity to in-circuit emulators, this approach does not scale with the growing system complexity. The design-for-test philosophy eliminated the need and cost for designated bond-out chips by integrating boundary scan chains, as introduced by the popular Joint Test Action Group standard.⁷ They enable valuable operational insight at a significantly reduced silicon cost for debugging purposes on a slow timescale. However, the continuous monitoring of a system running under real-time conditions stays far out of reach for this technique.

In contrast to Intel, Arm—with its strong background in embedded applications—embraced the idea of ET early. The evolution of their ETM protocols illustrates the steps of detaching from classic signal probing. ETMv1 uses two physical output channels. The three-bit *pipeline status* (PIPESTAT) reports the operation of the execution stage of the instruction pipeline on a cycle-by-cycle basis. Whereas this status channel closely resembles a signal probe, the trace packet port (TRACEPKT) communicates accompanying

address information as a paced data stream. Internal first-in, first-out buffers flatten bursts for gentler bandwidth requirements. The alignment of the trace data with the pipeline status must be inferred. The status reveals the presence and type of trace packets. Some status bits communicated in the delay slots after an executed branch are repurposed to convey the current slack between these two information flows. ETMv2 introduces packet headers for a standalone inline differentiation of packet types within the trace packet stream. ETMv3, finally, abolishes the pipeline status altogether and merges all trace communication into a single trace data stream. Designated P-headers aggregate the execution status of consecutive instructions efficiently.

More recently introduced trace protocols, in particular Arm PFT and Intel PT, focus on the concise communication of the program control flow. While the ETMv1-v3 protocols trace the execution of every single instruction, these protocols restrict their communication to control-flow diversions that are not already implied. Both forsake integral data-tracing capabilities. This strong focus makes these ET implementations scale more easily with the performance of modern processor systems. They are, nonetheless, tradeoffs that limit observability. Some mitigation may be offered through lightweight user instrumentation. Both the Arm instrumentation trace module (ITM) and the PTWRITE mechanism of Intel PT enable an application to inject user data into the trace stream at runtime using very cheap instructions, which particularly avoid competing over software-implemented logging facilities. However, these mechanisms are not supported by all trace-enabled processors.

Also, the evolution of the ETM protocol family continues. ETMv4 has dropped the strict tracing of every single instruction in favor of ideas adopted from PFT. Nonetheless, it takes a more holistic approach covering the execution status of conditional nonbranch instructions and including data trace. The availability of these features is implementation dependent; their activation is optional. Another remarkable feature of the ETMv4 protocol is its capability to communicate the completion or cancellation of speculatively executed code.

Further proposals for the trace data reduction come from the scientific community. Milenković et al.⁸ propose to target repetitions, as they are found in the trace of an iterating loop, by classic compression techniques. While this promises a significant reduction of the average trace bandwidth, it also demands a significant investment in on-chip buffer capacity both for flattening remaining trace peaks and for providing the trace history to facilitate the compression. No commercial implementation has yet committed to this tradeoff. These rather remain limited to variations of delta compression. The compression of the instruction pointer is the ubiquitous example shared across all trace protocols. Hochberger and Weiss⁹ propose a completely different approach to monitoring a processor. They synchronize an external emulation of the CPU core (digital twin) using runtime information. This much more easily accessible replica enables a complete system observation and analysis. It exposes internals such as stack pointers or register values, which typically remain hidden when using traditional trace approaches. The required bandwidth here no longer depends on the trace

protocol but on the effort required for synchronization. However, the emulated twin eventually faces similar scaling bounds as a bond-out chip.

TRACE DATA PROCESSING

Figure 3 illustrates the three main options for processing trace data. Option 1 is the trace data capture in system memory. This solution is often used in desktop environments. However, this setup impacts the system behavior significantly as the trace capture competes for shared system resources with the monitored application. Avoiding this behavioral feedback, trace data can also be captured by external tools via a trace interface. Feeding traditional trace tools establishes Option 2. They are essentially designated large memory buffers that collect the trace data for their later offline processing.

Both of these buffering approaches share two critical disadvantages. First, the observation time is strictly bounded by the buffer size. Depending on the trace bandwidth, it may allow trace snapshots of a few milliseconds or, at

most, seconds. These severely constrained timeframes limit the value of ET for use cases that depend on a comprehensive picture of an application’s execution. Second, the offline processing of recorded traces implies a long latency between observation and perception. This is not only an inconvenience for engineers, but it also precludes any innovative exploitation of the system observability for prompt responses.

Recent smart trace tool solutions¹⁰ feature the online analysis of ET as shown by Option 3. The large buffer memory for decoupling the offline trace processing is no longer needed. However, this approach is still challenged by the bandwidth of the trace source. Trace decompression and control-flow reconstruction must regularly cope with the execution traces from multiple CPUs that are running at nominal clock speeds above 1 GHz. A particular challenge is the management of the context information that must be extracted from an application binary for interpreting its execution trace. The online processing demands

the prompt availability of this information upon a context switch. With no time for an on-demand generation, fast lookup representations of the control-flow graphs of monitored applications must be generated ahead of time. Nonetheless, the online analysis approach is the right tradeoff for debugging and monitoring an embedded CPS performing a known set of tasks since it removes all limitations in terms of observation time and latency.

The real-time availability of the reconstructed program flow enables various relevant back-end tasks including 1) the recording of branch information for a test coverage analysis, 2) the validation of observed event sequences against a formal temporal logic specification, 3) the measurement of basic block execution times for a hybrid worst-case execution time (WCET) analysis,¹¹ or 4) the tracking of critical response times (WCRT). All of these are helpful building blocks for preventing and dissecting complex nondeterministic software anomalies.

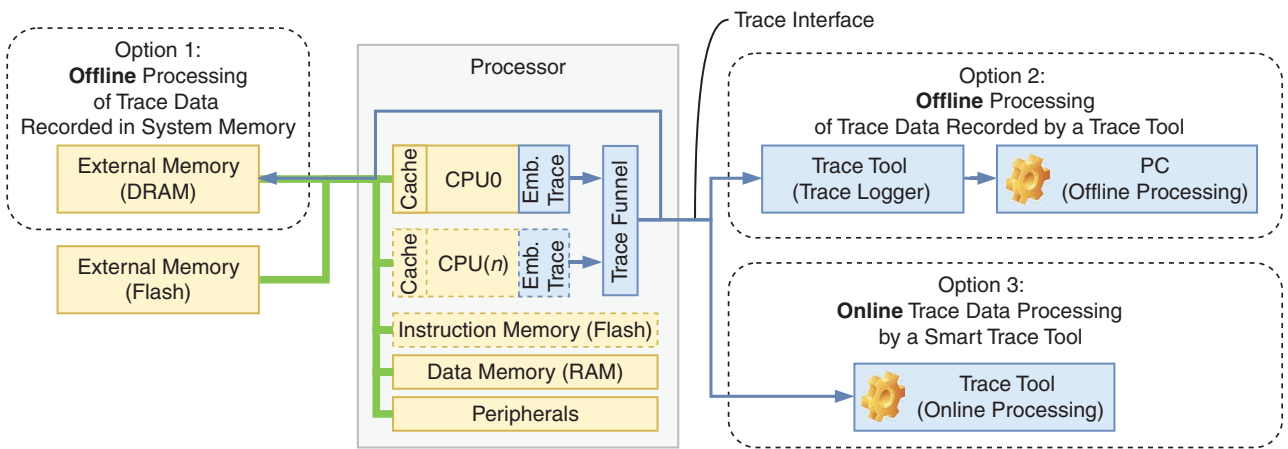


FIGURE 3. Trace data buffering and processing.

Both external trace options need a physical port to carry the trace data. Solutions relying on designated trace interfaces are increasingly converging toward either the parallel or the high-speed serial Mobile Industry Processor Interface (MIPI) alliance interface standards: parallel trace interface (PTI) and high-speed trace interface (HTI). Arm high-speed serial trace probe (Arm HSSTP) is an instance of the latter. On the other hand, the use of standard interfaces such as USB, PCIe, and GbEth is growing in popularity. Intel's current trace implementations make this choice.

USE CASE: CODE COVERAGE TRACKING

Consider the coverage analysis of high-level integration and system tests. ET provides all of the execution data needed to determine which code is executed by a test and which is not. A trace capture of merely a few seconds may suffice to collect these data when

testing systems with a small functional scope, controlling processes on an equally short timescale. However, this solution scales poorly as the required trace storage capacity grows proportionally with the desired observation time. The online analysis, on the other hand, immediately reduces the trace data into annotations of the control-flow graph of the application under test. Only its size determines the required backing storage irrespective of the runtime of the test.

Be aware that ET reports the program flow followed by the CPU through the control-flow graph of the executed binary rather than the source code. The structural correspondence between the two may be complicated, particularly, by optimizing compiler passes. Common code is hoisted out of branches, invariants are hoisted out of loops. Loop controls may migrate from head to foot or vice versa. Even, entire control-flow edges may disappear, for example, 1) by the inference of higher level arithmetic

instructions as for computing the minimum or maximum, or 2) by combining multiple branching conditions arithmetically before performing a conclusive jump. Likewise, control-flow edges may be introduced as the compiler is adding code to handle corner cases. Thus, extra branches are introduced regularly for handling Not a Number (NaN) values in floating-point comparisons.

Compilers can assist relating object code to corresponding source code by emitting debug information. It is typically provided in the DWARF format and conveys plenty of information including, for example, the association of source code with object code address ranges, and the local mapping of source code variables to registers and memory locations. As illustrated in Figure 4, this DWARF debug information enables the visualization of a traced program execution in terms of the source code. The left-hand side of the figure shows the actual trace observation annotating

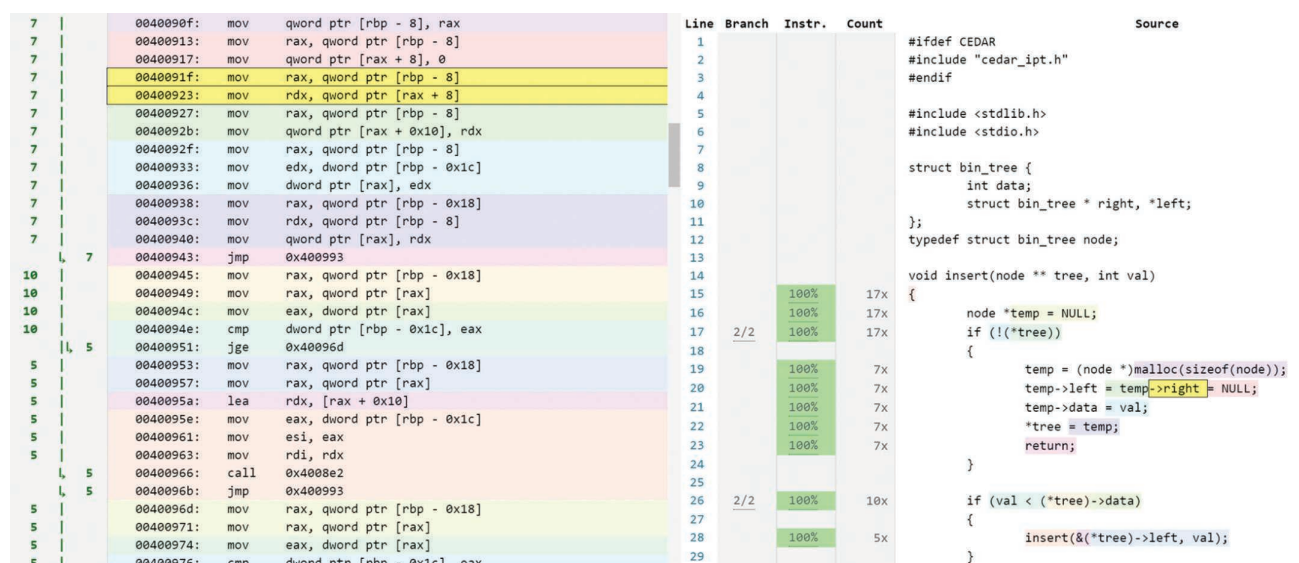


FIGURE 4. Object code coverage with source code mapping.

ABOUT THE AUTHORS

THOMAS B. PREÜBER heads the research team at Accemic Technologies, Dresden, 01217, Germany. His research interests include massively parallel compute acceleration and data processing in heterogeneous FPGA-based systems as well as computer arithmetic. Preüßer received a Ph.D. (Dr.-Ing.) in computer engineering from TU Dresden. He is a member of ACM. Contact him at tpreusser@accemic.com.

SMITHA GAUTHAM is a postdoctoral scholar at Virginia Commonwealth University, Richmond, Virginia, 23284, USA. Her research interests include the design and assessment of heterogeneous runtime verification architectures, runtime safety and security monitors for cyberphysical systems, and model-based design assurance and verification. Gautham received a Ph.D. in electrical engineering from Virginia Commonwealth University. Contact her at gauthamsm@vcu.edu.

ABHI D. RAJAGOPALA is a postdoctoral scholar at Virginia Commonwealth University, Richmond, Virginia, 23284, USA. His research interests include high-performance and reconfigurable system design, CPU and hardware accelerator architectures, and hardware realization of safety-critical systems. Rajagopala received a Ph.D. in electrical engineering from the University of North Carolina. He is a Member of IEEE. Contact him at rajagopalaad@vcu.edu.

CARL R. ELKS is an associate professor at Virginia Commonwealth University, Richmond, Virginia, 23284, USA. His research interests include the analysis, design, and assessment of dependable embedded cyberphysical systems of the type found in critical infrastructure. Elks received a Ph.D. in electrical engineering from the University of Virginia. He is a Member of IEEE. Contact him at crelks@vcu.edu.

ALEXANDER WEISS is the cofounder and chief executive officer of Accemic Technologies, Kiefersfelden, 83088, Germany. His research interests include cyberphysical systems runtime analysis, software verification, functional safety, and cyberattack detection. Weiss received a Ph.D. (Dr.-Ing.) in computer science from TU Dresden. Contact him at aweiss@accemic.com.

enhance their dependability. We discuss how ET has become an enabling technology for the verification of CPSs in real time. ET addresses the age-old problem of the intrusiveness of fine-grain program observability based on instrumentation. We illustrate the challenging information density of ET and describe the development of increasingly efficient engineering solutions. We demonstrate how online trace processing is able to access new powerful use cases that remained infeasible with a traditional decoupled trace capture. We plan to investigate further the utilization of ET to achieve a Dependable DevOps (Development and Operation) continuum where ET can be used during both development and operation. The DevOps approach aims to carry forward the assurance of safety and security from design time to runtime in a systematic way. 

ACKNOWLEDGMENTS

The authors acknowledge the financial support by the Federal Ministry of Education and Research of Germany in the KMU Innovative project CoCoSI (Project 01IS19044). Other parts of this work were supported by the Virginia Commonwealth Cyber Initiative USA, an investment in the advancement of cyber R&D, innovation, and workforce development. For more information, visit www.cyberinitiative.org.

REFERENCES

1. A. Weiss *et al.*, "Understanding and fixing complex faults in embedded cyberphysical systems," *Computer*, vol. 54, no. 1, pp. 49–60, 2021, doi: 10.1109/MC.2020.3029975.
2. A. Goodloe, "Challenges in high-assurance runtime verification," in *Proc. Int. Symp. Leveraging Appl. Formal Methods*,

each instruction with its execution count. The right-hand side transfers this information back into the realm of the programmer who can, thus, understand hot execution paths or coverage issues within tests.

With the advent of increasingly complex advanced embedded systems and their use in safety-critical applications, design and runtime verification are becoming important concerns to

- Springer-Verlag, 2016, pp. 446–460, doi: 10.1007/978-3-319-47166-2_31.
3. “CoreSight™ Program Flow Trace™,” ARM, 2011. [Online]. Available: <https://developer.arm.com/documentation/ih0035/>
 4. “Embedded Trace Macrocell ETMv1.0 to ETMv3.5,” ARM, 2011. [Online]. Available: <https://developer.arm.com/documentation/ih0014/>
 5. “Intel® 64 IA-32 architectures software developer’s manual,” Intel, Apr. 2021. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
 6. *The Nexus 5001 Forum - Standard for a Global Embedded Processor Debug Interface*, IEEE Standard 5001-1999, Jun. 2012.
 7. *IEEE Standard for Reduced-Pin and Enhanced-Functionality Test Access Port and Boundary-Scan Architecture*, IEEE Standard 1149.7-2009, 2010.
 8. A. Milenković, V. Uzelac, M. Milenković, and M. Burtscher, “Caches and predictors for real-time, unobtrusive, and cost-effective program tracing in embedded systems,” *IEEE Trans. Comput.*, vol. 60, no. 7, pp. 992–1005, Jul 2011, doi: 10.1109/TC.2010.146.
 9. C. Hochberger and A. Weiss, “Acquiring an exhaustive, continuous and real-time trace from SoCs,” in *Proc. Int. IEEE Conf. Comput. Design*, 2008, pp. 356–362, doi: 10.1109/ICCD.2008.4751885.
 10. “Cedartools® – Look inside your processor,” Accemic Technologies, 2020. [Online]. Available: <https://accemic.com/cedartools/>
 11. D. Kästner, M. Pister, S. Wegener, and C. Ferdinand, “TimeWeaver: A tool for hybrid worst-case execution time analysis,” in *Proc. 19th Int. Workshop Worst-Case Execution Time Anal. (WCET 2019)*, 2019, vol. 72, pp. 1:1–1:11.



IEEE COMPUTER SOCIETY
Call for Papers

Write for the IEEE Computer Society's authoritative computing publications and conferences.

GET PUBLISHED
www.computer.org/cfp


IEEE COMPUTER SOCIETY
